

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

A PRIVACY-PRESERVING EDGE TINYML SYSTEM FOR ACOUSTIC COMFORT ESTIMATION USING HYBRID DSP–CNN FUSION AND VISUAL FEEDBACK

Submitted By:

Plan Ghimire (080BEI028)

Pranjal Shrestha (080BEI033)

Sagun Acharya (080BEI037)

Samman Ghale (080BEI039)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing the academic environment, resources and guidance required to prepare this project progress report.

We are deeply thankful to our supervisor, Er. Binod Sapkota, for his continuous guidance, valuable suggestions, and encouragement throughout the development of this project and the preparation of this report. His insights have been instrumental in helping us navigate the technical challenges of our implementation.

We also extend our appreciation to our teachers, internal evaluators, and colleagues for their constructive feedback and recommendations during our progress reviews, system refinement, and defence preparation.

This project addresses the practical problem of acoustic disturbance in shared spaces. By implementing embedded technologies such as the ESP32-S3, INMP441 I2S microphone, digital signal processing, embedded deep learning and IoT dashboards, we aim to build a feasible, privacy-preserving and product-oriented system. The feedback received during our progress discussions has been vital in refining our methodology and guiding our next steps toward final implementation.

Project Team

ABSTRACT

This project is a privacy-preserving, edge-based smart acoustic comfort indicator that classifies acoustic behavior in classrooms and shared workspaces into intuitive visual states such as Smooth, Pre-Warning, Warning, Loud, Recovery and Spike. It aims to mitigate auditory distractions in working and learning environments by providing non-judgmental visual feedback on the room's acoustic state. The planned hardware platform consists of an ESP32-S3 microcontroller, an INMP441 I2S MEMS microphone, dual status displays, custom visual indicators, a battery charging circuit, speaker support and Wi-Fi connectivity. The submitted firmware calculates A-weighted loudness, FFT-based DSP features, speech-band energy, peak events and a 49 × 64 Log-Mel feature tensor locally. A compact, fully quantized INT8 DS-CNN and digital signal processing (DSP) are then used to classify the acoustic state on-device. Only anonymous scalar metrics are exposed through the local dashboard, ensuring user privacy, while MQTT-based remote telemetry remains a planned extension. The expected outcome is a functional, product-level prototype featuring real-time audio analysis, privacy-preserving data processing, remote dashboard visualization and a comprehensive validation report.

Keywords: ESP32-S3, INMP441, A-weighting, FFT, Log-Mel, Embedded Deep Learning, CNN, TensorFlow Lite Micro, MQTT, Acoustic Comfort, Privacy by design

TABLE OF CONTENTS

ACKNOWLEDGEMENT i

ABSTRACT ii

TABLE OF CONTENTS iii

LIST OF FIGURES v

LIST OF TABLES vi

LIST OF ABBREVIATIONS vii

1. INTRODUCTION 1

1.1 Background 1

1.2 Motivation 2

1.3 Problem Definition 3

1.4 Objectives 3

1.5 Scope and Applications 4

2. LITERATURE REVIEW 5

2.1 Irrelevant Speech, Lombard Effect and Acoustic Comfort 5

2.2 Acoustic Standards, Speech Levels and Threshold Selection 5

2.3 A-Weighting and Digital Signal Processing 6

2.4 Voice Activity Detection and Embedded Audio Features 6

2.5 Transfer Learning and Knowledge Distillation Theory 8

2.6 Embedded CNN and TinyML Deployment 9

2.7 IoT-Based Noise Monitoring Systems 10

2.8 MQTT Communication and Privacy-Preserving Technique 10

2.9 Comparative Gap Analysis 11

3. METHODOLOGY 13

3.1 Overall Hybrid System Architecture 13

3.2 Hardware Pin Mapping and Interface Schematic 14
3.3 Real-Time Firmware Task Pipeline 15
3.4 Mathematical Conversions: I2S to Calibrated dBA 15
3.5 State Transition Architecture 16
3.6 Temporal Decision Engine and Scenario Timing Diagrams 17
3.7 Dataset Preparation and Machine Learning Training 22
3.8 Output Behavior and UI Design 23
3.9 Privacy Boundary Architecture 24
3.10 Instrumentation Tools 25
3.11 Dashboard Instrumentation and Diagnostic Telemetry 25
4. EXPECTED OUTCOME AND VALIDATION PLAN 30
4.1 Real-World Validation Scenarios 30
4.2 Physical Measurement Accuracy Calibration 31
4.3 Machine Learning Validation 31
5. PROJECT SCHEDULE 32
6. PROJECT BUDGET 33
7. FEASIBILITY ANALYSIS 34
7.1 Limitations 34
7.2 Risk Management 34
7.3 Future Enhancements 35
REFERENCES 36

LIST OF FIGURES

Figure 1: Overall system block diagram 13
Figure 2: Standard Score Escalation and Stepped Decay 20
Figure 3: Loud State timing diagram 21
Figure 4: Hardware Fault Lockout Timing Diagram 21
Figure 5: Privacy boundary architecture 24

LIST OF TABLES

Table 1: Comparative Gap Analysis of Prior Works and Proposed System.....12
Table 2: ESP32-S3 hardware pin mapping and interface allocation 14
Table 3: Current firmware task allocation 15
Table 4: FSM Waterfall Priority Transition Rules 17
Table 5: Confusion Matrix before quantization.....23
Table 6: Confusion Matrix after quantization.....23
Table 7: Instrumentation Tools.....25
Table 8: Validation Scenario Matrix.....30
Table 9: Project timeline.....32
Table 10: Project budget.....33
Table 11: Feasibility analysis summary.....34

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADC	Analog to Digital Converter
CNN	Convolutional Neural Network
dBA	A-weighted decibel
DMA	Direct Memory Access
DSP	Digital Signal Processing
FFT	Fast Fourier Transform
GPIO	General Purpose Input Output
I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
IoT	Internet of Things
LAeq	Equivalent Continuous A-weighted Sound Level
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MFCC	Mel-Frequency Cepstral Coefficients
MQTT	Message Queuing Telemetry Transport
OLED	Organic Light Emitting Diode
PCM	Pulse-Code Modulation
RAM	Random Access Memory
RMS	Root Mean Square
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TinyML	Tiny Machine Learning
TFLM	TensorFlow Lite for Microcontrollers
UI	User Interface
Wi-Fi	Wireless Fidelity

1. INTRODUCTION

1.1 Background

Offices and classrooms are environments where there are many people huddled together in a single room or a close environment. While it is necessary to maintain a quiet environment so everyone else in the room can focus without any distractions, this may not be fully possible due to it being hard to gauge how loud one is being, especially in already noisy environments where background noise covers up speech.

Human interaction is also usually necessary in these environments in order to share ideas and amplify the understanding of whatever is being discussed. This introduces a practical problem: how to recognize how loud one is being and how to mitigate the noise without creating more noise in turn or passing on an authoritative tone that may create a stressful environment unnecessarily.

The performance of people working in any shared environment with task-irrelevant speech, that is, speech that may not be relevant but is nonetheless present intelligibly in the background, decreases significantly even more than continuous noise at the same sound level [1]. The Speech Transmission Index measures speech intelligibility, and an index above 0.6 indicates that the semantics are largely understandable. Haka et al. observed that long-term memory recall decreased in conditions with higher speech intelligibility [2]. Irrelevant speech was also observed to increase subjective workload as well as overall stress of the people subjected to it [3]. The cognitive load generated by this processing is highly dependent on the linguistic structure of the noise, a phenomenon known in cognitive psychology as the Irrelevant Speech Effect.

In shared spaces, there can be presence of numerous feedback loops relating to human speech. The Lombard effect, first documented by Étienne Lombard in 1911, describes the involuntary and subconscious tendency of people to increase the level of their speech in a noisy environment in order to keep speech intelligible [4]. It has been observed that the Lombard effect can be triggered when environmental noise increases, leading speakers to increase their speech level by several decibels for every 10 dB increase in background noise [5], [6].

Especially in shared spaces, a feedback loop is observed between multiple speakers who need to speak simultaneously to maintain intelligibility for their respective

listeners. This effect, seen in shared spaces such as classrooms, cafés and canteens, is often understood as a café or canteen effect [7], [8]. As more people enter a single space and start talking with each other, their voices add to the background noise. As the background noise increases, the Lombard effect causes other people to increase their own voice levels. This localized increase in vocal effort causes the background noise to increase even further, creating a positive feedback loop until the room reaches high noise levels.

1.2 Motivation

Current monitoring methods for noise levels include active human policing, commercial numerical decibel meters or loud auditory alarms. These methods introduce various problems such as authoritative and social friction, demanding focal attention, or creating more stress and noise.

Traditional sound meters provide a numerical value for the noise being created in an environment. These devices are mainly created as engineering tools and feature rapidly changing abstract values. A person in a working environment cannot cognitively direct their focus onto the numerical value without breaking complete focus on the current task. As a result, these devices can become another source of distraction rather than an effective measure to reduce noise.

Many commercial noise warning systems try to solve constant monitoring through addition of a buzzer component that warns users when the noise level becomes too high. However, from a psychoacoustic standpoint, this approach is flawed and counterproductive because it adds sound to an already loud room. A buzzer may startle occupants, increase stress and break concentration, which is the opposite of the project objective.

To mitigate these issues and make sure the outcome is productive, this project aims to utilize slow, adaptive and color-shifting LED animations. This is done to leverage the human peripheral visual system, which is optimized to detect changes in color, motion and light intensity over a wide field of view. Such visual feedback requires much less cognitive concern than constantly interpreting changing numerical values and is less intrusive than a buzzer.

1.3 Problem Definition

The main problem this project tries to solve is the noise problem in classrooms as well as shared workspaces. The Lombard effect states that noise created in order to combat already existing noise levels is usually involuntary and subconscious. This means that participants may not be aware of the noise they are creating themselves.

The project also aims to mitigate the paradoxical cascading noise sources that are created while trying to mitigate previous noise. Usually, supervisors or peers in the area need to speak up and increase the noise to make sure the noise level is decreased. In addition, a person commanding others to reduce noise may put forward an authoritative tone and may induce or increase stress on the people who were unaware of the noise being made.

Currently, there is a lack of an objective, real-time mechanism to break the subconscious feedback loop. While human supervisors are able to intervene, their authority can be a stress inducer and may also disrupt the flow of the shared space. The fundamental problem in these scenarios is the absence of an objective, non-judgmental and non-human arbitrator in shared spaces.

By depersonalizing the feedback loop, an automated visual indicator serves as an impartial, non-human mediator. Because the system is objective and treats all occupants equally, it removes the personal judgment and defensive triggers associated with human-to-human policing, allowing occupants to self-regulate their volume naturally and maintain a collaborative, low-stress environment.

1.4 Objectives

To ensure that the problems are addressed correctly, the following objectives need to be fulfilled:

- To design, develop and validate a privacy-preserving edge-based smart acoustic comfort indicator based on a hybrid processing pipeline consisting of digital signal processing and custom embedded deep learning.
- To provide real-time, non-invasive visual feedback as well as anonymous cloud analytics for acoustic comfort in classrooms and workplaces.

1.5 Scope and Applications

The initial phase of the project targets a product-like integration of hardware, digital signal processing, embedded deep learning and dashboard visualization. The noise level is captured by an INMP441 I2S MEMS microphone and classified using a compact embedded deep learning model with a rule-based DSP fallback. The planned visual feedback will be delivered through dual status displays (LCD and OLED) and an LED ring placed inside a suitable enclosure. The submitted firmware already performs the audio, DSP, TinyML, fusion-state and local dashboard functions; remote telemetry and the physical indicators remain planned integration work.

The application of this project is versatile, ranging from libraries, classrooms, and quiet study spaces to corporate offices and collaborative zones. It can be deployed in any shared environment where acoustic comfort, concentration, and a collaborative atmosphere are essential. The device is intended strictly as a qualitative, behavioral coaching indicator rather than a certified, legally standard sound level meter.

The system processes audio entirely on the edge, maintains a local web dashboard and is designed to operate within standard classroom and office dimensions. The local dashboard requires a Wi-Fi-capable client, while DSP calculations and state transitions operate entirely offline; the planned visual indicators will also operate locally. To preserve user privacy, the system does not record, reconstruct or store raw audio signals at any stage.

2. LITERATURE REVIEW

2.1 Irrelevant Speech, Lombard Effect and Acoustic Comfort

Acoustic comfort does not equate to silence. A productive acoustic environment in an office or classroom is a space where the sound level of speech and activities matches the function of the room. While complete silence may not be applicable to a shared space, intelligible background speech can be disruptive during focused work. Research on the irrelevant speech effect has found that intelligible background speech distracts from memory processes and negatively affects performance even when the overall sound level is relatively low [1]-[3].

In this case, the Lombard effect serves as a physiological explanation of how a room can gradually get louder. With the increase in the amount of background sound, speakers begin to automatically speak louder to keep the intelligibility level [4]-[6]. In multi-speaker environments, this becomes a feedback loop: one loud group causes others to speak louder, which increases the background level and encourages further vocal effort. This loop is a key reason why this project focuses on gentle feedback rather than direct punishment or alarms.

2.2 Acoustic Standards, Speech Levels and Threshold Selection

As for the acoustic threshold implemented in this project, it is justified by the existing guidelines on acoustic comfort in classrooms and office settings. ISO 22955:2021 defines the quality of sound in open offices and highlights speech disturbance, discretion, intelligibility, and the Lombard effect in the context of shared workplaces [9]. Similarly, the strict guidelines in ANSI/ASA S12.60 are aimed at creating a comfortable acoustic environment for education [10].

This project does not seek to replace the use of a Class 1 or Class 2 sound level meter. Hence, the threshold values are applied as operating limits for a behavioral marker. Human conversations at approximately one meter generally measure around 55 to 65 dBA, while raised voices can reach 70 dBA [11]. As such, 55-70 dBA can form the basis for the conversational warning zone, and greater than 70 dBA can be applied for louder speech state limits. Additionally, this is done with CNN confidence to prevent false alarms on other sources generating the same level of sound.

2.3 A-Weighting and Digital Signal Processing

For testing environmental noise, measurement should reflect human hearing perception. Raw microphone signals may overrepresent low-frequency sources such as air-conditioning systems, ventilation units and traffic rumble. Rimell et al. developed implementation approaches for A-weighting and C-weighting filters using international acoustic standards [12]. A-weighting reduces very low and very high frequencies while emphasizing the range where the human ear is more sensitive, making it a common basis for environmental and occupational noise measurement.

However, this process is simply a method of applying a frequency response. It cannot differentiate between speech and non-speech sounds. Two sounds can be of equal intensity but come from different sources, such as conversations, typing, hand claps and machine operation. Consequently, this project implements A-weighting alongside FFT analysis, speech-band analysis, Log-Mel features and TinyML classification.

The Fast Fourier Transform is an efficient algorithm that converts time-domain signals into frequency-domain signals. The algorithm proposed by Cooley and Tukey minimizes the computational complexity of directly computing the Fourier transform [13]. Most workplace speech occupies the bandwidth from 300 Hz to 3400 Hz, while many steady noise sources occupy lower frequencies. This project uses FFT-derived speech-band metrics and a Log-Mel feature set to help distinguish conversations from low-frequency background noise.

2.4 Voice Activity Detection and Embedded Audio Features

2.4.1 Log-Mel Spectrogram vs. MFCC Feature Selection

Standard speech processing often relies on Mel-Frequency Cepstral Coefficients (MFCCs) to compress audio features. Generating MFCCs requires applying a Discrete Cosine Transform (DCT) to a Log-Mel spectrogram. This decorrelates the frequency bands. While decorrelation helps simpler linear classifiers, it harms Convolutional Neural Networks (CNNs) [15].

CNNs learn patterns by looking at local spatial correlations: neighboring bins in both time and frequency [15]. Applying a DCT discards this spatial structure. It destroys vocal formant transitions and acoustic textures. Retaining the raw Log-Mel

spectrogram as a 2D time-frequency grid preserves these patterns, allowing the convolutional layers to extract edge features and spectral resonances directly [15].

2.4.2 Feature Extraction Geometry and Input Dimensionality

To fit the microcontroller's memory limit while retaining high spectral detail, the input features map to a fixed 2D grid. The signal is sampled at 16 kHz mono [11]. A 512-point Fast Fourier Transform (FFT) runs using a Hann window with a 320-sample hop length [13]. The resulting power spectrum projects onto 64 Mel bins spanning 80 Hz to 7600 Hz. Over a 1-second audio window (16,000 samples) [14], the exact number of time frames is:

$$Time_Frames = \left\lfloor \frac{N_{samples} - N_{fft}}{N_{hop}} \right\rfloor + 1 = \left\lfloor \frac{16000 - 512}{320} \right\rfloor + 1 = 49$$

This geometry yields a 2D input tensor of [49 × 64 × 1] for every 1-second audio frame.

2.4.3 Per-Clip Mean and Per-Mel Normalization

To reduce sensitivity to microphone gain and physical distance while retaining spectral structure, the feature extraction pipeline applies scalar per-clip mean subtraction followed by per-Mel standardization:

$$\mu_{clip} = \frac{1}{TF} \sum_{t=1}^{T} \sum_{f=1}^{F} X_{logmel}(t, f)$$

$$X_{norm}(t, f) = \frac{X_{logmel}(t, f) - \mu_{clip} - \mu_f}{\sigma_f}$$

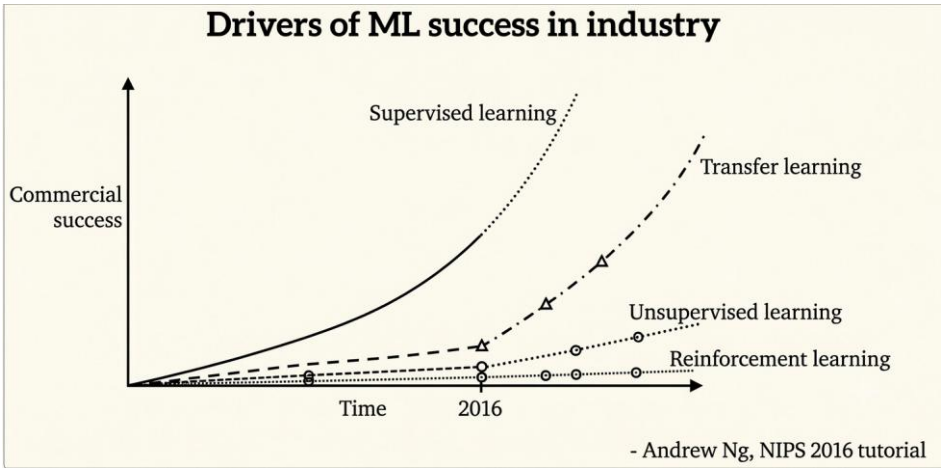
Here, μ_clip is the scalar mean across all 49 frames and 64 Mel bins, while μ_f and σ_f are the exported per-Mel training-set statistics. The firmware first subtracts the clip mean and then applies per-Mel standardization. This reduces global volume dependence while retaining spectral and temporal structure, allowing the machine-learning path to classify sound context while absolute dBA severity remains in the deterministic DSP path [12].

2.5 Transfer Learning and Knowledge Distillation Theory

2.5.1 The Transfer Learning Paradigm in Audio

Training deep learning models from scratch requires massive datasets. The network needs thousands of hours of audio just to learn basic acoustic properties like pitch, timbre, and temporal envelopes. For custom embedded applications, such datasets are not available. Transfer learning solves this by taking a model pre-trained on a massive dataset and adapting its learned representations to the target task [20].

Here, a frozen YAMNet model serves as the foundation. YAMNet is a MobileNetV1 architecture trained on millions of AudioSet clips [15]. For each 1-second clip, YAMNet extracts 1024-dimensional embeddings [15]. Calculating and concatenating the mean and standard deviation of these embeddings along the time axis yields a 2048-dimensional feature vector. A lightweight classification head trained on these vectors achieves 96.06% accuracy on the test split. This bypasses the need for a massive local training dataset.



2.5.2 The Teacher-Student Knowledge Distillation Framework

Running YAMNet directly on an embedded microcontroller is unfeasible due to its size. Instead, Knowledge Distillation transfers the predictive power of this heavy "teacher" model to a compact "student" model [20].

The student is trained to match the softened probability outputs of the teacher rather than just hard binary labels. The teacher's final Softmax layer is softened using a temperature scaling parameter (T = 2.0):

$$P_i = \frac{e^{\frac{z_i}{T}}}{\sum_j e^{\frac{z_j}{T}}}$$

where z_i is the logit for class i. This softening reveals the hidden similarities between classes. It shows the student that a specific speech clip also shares acoustic similarity with non-speech noise. Providing this soft target distribution prevents the student from overfitting and allows a highly compressed model to generalize well on inexpensive hardware [19].

2.6 Embedded CNN and TinyML Deployment

2.6.1 Compact Student DS-CNN Architecture

To keep inference latency near zero on the edge, the student model uses Depthwise Separable Convolutions (DS-CNN) [19]. Standard convolutions filter spatial data and mix channels in a single step, which is computationally expensive. DS-CNN splits this into two steps. A 3x3 depthwise convolution filters spatial features per channel. A 1x1 pointwise convolution then mixes the channels together [19]. This cuts computation and memory requirements by over 80%.

The student model contains only 46,867 parameters (183.07 KB) [19]. It is structured sequentially:

1. Input Layer: [49 x 64 x 1] Log-Mel spectrogram.
2. Stem Layer: Conv2D (32 filters, 3x3, stride 2, no bias), BatchNorm, ReLU6.
3. Block 1: DS-Conv (48 filters, stride 1), BatchNorm, ReLU6.
4. Block 2: DS-Conv (64 filters, stride 2), BatchNorm, ReLU6.
5. Block 3: DS-Conv (96 filters, stride 2), BatchNorm, ReLU6.
6. Block 4: DS-Conv (128 filters, stride 2), BatchNorm, ReLU6.
7. Block 5: DS-Conv (128 filters, stride 1), BatchNorm, ReLU6.
8. Output Stage: GlobalAveragePooling2D, Dropout (20%), Dense (3 classes), Softmax.

The compiled model is fully optimized for TensorFlow Lite Micro (TFLM) execution on the target hardware platform [20].

2.6.2 Full INT8 Post-Training Quantization (PTQ)

To avoid floating-point math overhead on the hardware, the model is converted to signed 8-bit integers (INT8) using Post-Training Quantization (PTQ) [20].

The converter uses a representative dataset of 250 samples per class to record activation ranges. It optimizes the scale (S) and zero-point (Z) parameters for every tensor [20]:

$$q = round(r/S) + Z$$

Forcing both inputs and outputs to signed int8 enables fast integer-only calculations [20].

2.7 IoT-Based Noise Monitoring Systems

Marques and Pitarma proposed an economical IoT noise logger solution for real-time monitoring of acoustic comfort, based on sound level sensing and Wi-Fi dashboards, yet without applying machine learning and speech-noise separation [16]. It showcases the application of IoT dashboards for acoustic comfort, and also highlights the problem of dBA-only monitoring.

Hammad et al. showed the applicability of TinyML and anomaly detection algorithms to detect acoustic events in near-zero latency and low power consumption on microcontrollers [17]. Such research gives ground for applying learning on devices in order to process data privately. But, the problem of anomaly detection does not tell us what kind of sound leads to that anomaly. This project extends in that direction by classifying acoustic states into several classes and employing a visual feedback mechanism.

2.8 MQTT Communication and Privacy-Preserving Technique

The submitted firmware currently exposes anonymous scalar metrics through a board-hosted local HTTP dashboard and JSON status API. MQTT is planned as a remote-telemetry extension. In the publish-subscribe architecture of MQTT, clients are separated by a broker: the edge device publishes to a topic, while the dashboard service subscribes to obtain the data [25], [26].

MQTT is planned because of its low-latency and efficient communication properties for lightweight telemetry. While HTTP communication involves request and response headers, MQTT uses small fixed-size headers and supports efficient message delivery

[25]. The planned MQTT packets will include only anonymous numeric information such as LAeq, state index and event counts. Raw audio packets will not be transmitted.

Employing audio sensors in shared workspaces raises many issues related to privacy, law, and ethics. Instead of avoiding such problems, traditional projects choose to store the collected data in the cloud for processing purposes. However, doing so could leak sensitive information. As a solution, new IoT and edge AI applications consider privacy-by-design paradigms where all data processing operations are handled locally, and raw data is stored in volatile memory [28].

2.9 Comparative Gap Analysis

Apart from previous work limitations, this project solves the limitation of prior research through a practical hybrid solution approach. Simple dBA logger devices record sound levels but are unable to detect whether human voice is mixed with fans, AC, door closure, or other noises in the workspace. Also, binary detectors can identify anomalies but lack state separation functionality that would be required by the indicator. Speaker recognizer devices utilize ESP32-S3 and INMP441 audio sensor capabilities; however, their purpose is speaker identification rather than environmental sound state feedback. This project combines DSP, TinyML, state-transition hysteresis, privacy preserving dashboard and visual feedback merged into a low-cost prototype.

Table 1: Comparative Gap Analysis of Prior Works and Proposed System

System / Prior Work	Reference(s)	Main Processing Approach	Main Strength	Limitation / Research Gap	How our System Addresses the Gap
Traditional decibel meter / sound-level measurement	[10], [11], [12]	Measures RMS or A-weighted sound level and displays numerical dB/dBA value	Useful for objective sound-level measurement and calibration	Does not identify sound source; no speech/noise separation; no adaptive comfort feedback	Uses calibrated LAeq as one input, but adds TinyML classification, state logic and visual feedback
IoT-based noise monitoring system	[16], [25], [26], [27]	Measures sound level and sends data to cloud or dashboard using IoT communication	Enables real-time monitoring, logging and dashboard visualization	Mostly dBA-based; limited local intelligence; privacy depends on transmitted data type	Current firmware exposes anonymous scalar metrics through a board-hosted local HTTP dashboard; remote telemetry remains planned
TinyML acoustic anomaly / audio event detector	[17], [19], [20], [24]	Uses lightweight embedded ML models for acoustic event or anomaly detection	Supports low-power edge inference and faster local detection	Often detects anomaly only; does not provide comfort-state feedback or stable behavioral transitions	Uses compact CNN/TinyML with DSP fusion, six acoustic states, timers and hysteresis
This project's proposed system	This work; supported by [20]–[28]	Hybrid fast DSP + INT8 CNN/TinyML on ESP32-S3 with local HTTP dashboard; MQTT planned	Combines fast physical metrics, sound-context classification, visual feedback design and privacy-preserving telemetry	Prototype requires calibration, dataset tuning and real-world validation	Provides an integrated privacy-preserving acoustic comfort indicator for classrooms and shared spaces

3. METHODOLOGY

3.1 Overall Hybrid System Architecture

The project system uses a hybrid real-time edge-computing architecture that runs locally on the ESP32-S3 microcontroller. To balance rapid response with accurate sound-context recognition, acoustic analysis is divided into two branches within the dedicated audio task: a deterministic Digital Signal Processing path and a probabilistic Convolutional Neural Network path.

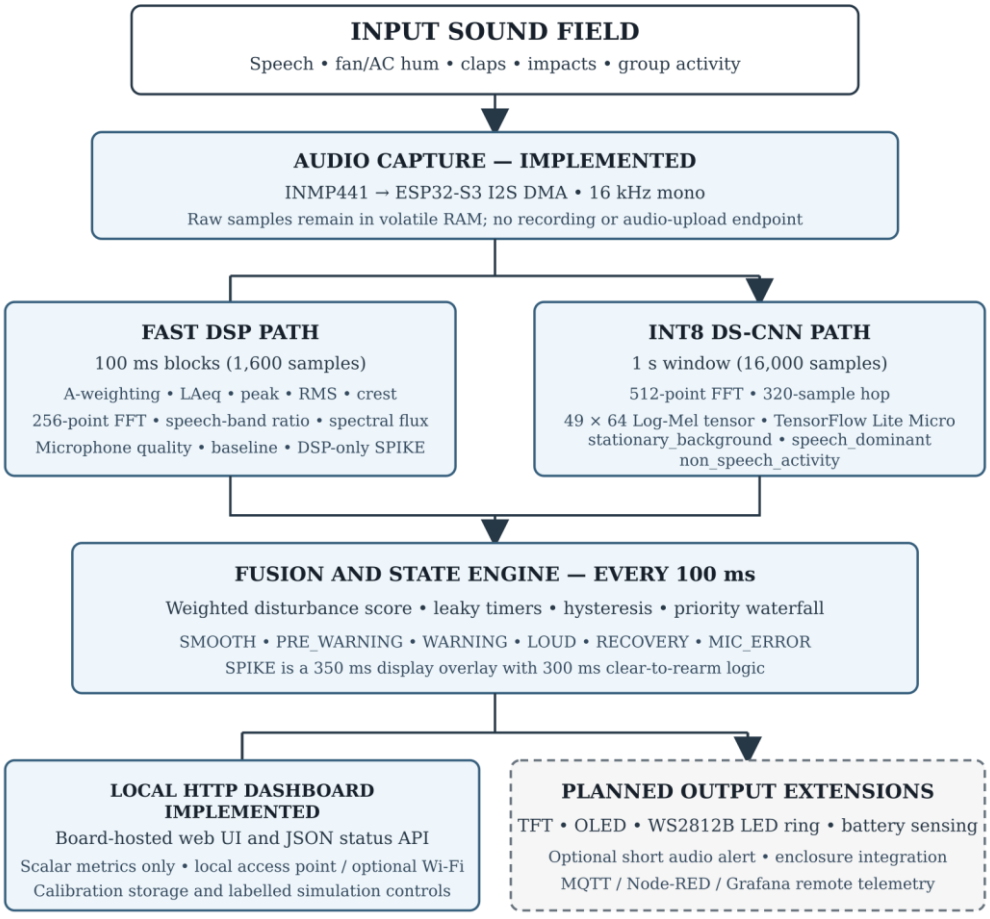


Figure 1: Overall system block diagram

The system begins from the physical input sound field, which may contain speech, fan/AC hum, claps, keyboard clicks, transient drops or loud group activity. The INMP441 microphone converts these sound pressure waves into 24-bit digital PCM data and transmits them through I2S to the ESP32-S3. The DMA controller stores samples in a volatile ring buffer in SRAM, ensuring that raw audio remains only in temporary memory.

The deterministic DSP path executes every 100 ms and extracts physical metrics such as RMS, LAeq, peak dB, speech-band ratio and spectral flux. The CNN/TinyML path executes on a one-second window and estimates confidence values for stationary_background, speech_dominant and non_speech_activity. LOUD and SPIKE remain DSP-only outputs. The hybrid decision engine combines the DSP metrics and CNN probabilities, applies confirmation timers and determines the final state. The submitted firmware exposes anonymous metrics through a local HTTP dashboard; physical display outputs and MQTT telemetry are planned extensions.

3.2 Hardware Pin Mapping and Interface Schematic

The submitted firmware implements the INMP441 I2S connections on GPIO 14, 15 and 16. The remaining allocations in Table 2 are planned or reserved for physical display, indicator, alert and battery-sensing integration and are organized to avoid bus conflicts.

Table 2: ESP32-S3 hardware pin mapping and interface allocation

Component	Protocol	Component Pin	ESP32-S3 GPIO	Electrical Description
INMP441 Microphone	I2S	SCK / BCLK	GPIO 14	Serial clock line for I2S audio
INMP441 Microphone	I2S	WS / LRCLK	GPIO 15	Word select / channel select
INMP441 Microphone	I2S	SD / DATA	GPIO 16	Serial data output from microphone
GC9A01 Circular TFT	SPI	SCL / Clock	GPIO 12	SPI serial clock
GC9A01 Circular TFT	SPI	SDA / MOSI	GPIO 11	SPI data line
GC9A01 Circular TFT	SPI	CS	GPIO 10	Chip select
GC9A01 Circular TFT	SPI	DC	GPIO 9	Data/command select
GC9A01 Circular TFT	SPI	RST	GPIO 13	Hardware reset
SSD1306 OLED	I2C	SCL	GPIO 1	I2C clock
SSD1306 OLED	I2C	SDA	GPIO 2	I2C data
WS2812B LED Ring	One-Wire/RMT	DIN	GPIO 48	LED data signal
Speaker / Buzzer	PWM or I2S Amp	Input	GPIO 4	Short alert sound driver
Battery Monitor	Analog ADC	VBAT Sense	GPIO 5	Voltage divider input

3.3 Real-Time Firmware Task Pipeline

The submitted firmware creates one dedicated FreeRTOS audio task pinned to Core 1. This task captures I2S audio, computes DSP metrics every 100 ms, runs the CNN once per 1-second window and updates the fusion state engine. The Arduino main loop services the local WebServer and Wi-Fi reconnection with a short non-blocking delay. This allocation reflects the current implementation while leaving the second core available to the Arduino networking runtime and future output integration.

Table 3: Current firmware task allocation

Core	Task	Priority	Period	Purpose
Core 1	AuraSyncAudio FreeRTOS task	Priority 3	Every 100 ms; CNN every 10 cycles	Captures I2S audio, computes DSP and CNN paths, and updates fusion/state logic
Arduino main-loop context	WebServer and Wi-Fi service	Default runtime context	Continuous; 2 ms loop delay	Serves the local dashboard/API and handles Wi-Fi reconnection

3.4 Mathematical Conversions: I2S to Calibrated dBA

To transform raw digital audio samples captured by the I2S peripheral into a human-audible sound level estimate, the ESP32-S3 executes a real-time digital signal processing pipeline. The raw INMP441 digital data are first bit-shifted, cleaned, windowed, transformed into the frequency domain and calibrated against a reference sound-level meter.

3.4.1 Bit-Shifting and DC Offset Removal

The INMP441 MEMS microphone transmits digital audio data over the I2S bus in a 32-bit slot width format, where the actual signed PCM data occupies the upper bits. The ESP32-S3 recovers the usable signed sample and converts it to a floating-point or fixed-point processing variable. Digital MEMS microphones introduce a static DC offset caused by internal analog-to-digital converter (ADC) biases and hardware misalignments. If left untreated, this zero-frequency offset dominates the low-frequency spectrum, corrupting downstream Root Mean Square (RMS) energy and Fast Fourier Transform (FFT) spectral estimations.

To actively track and eliminate this bias, a first-order Infinite Impulse Response (IIR) high-pass filter is implemented

$$y[n] = x[n] - x[n-1] + \alpha y[n-1] (\alpha = 0.995)$$

where x[n] is the raw input sample from the I2S DMA buffer, y[n] is the filtered output with the DC offset removed, and α = 0.995 is the high-pass filter coefficient used by the firmware.$x[n]y[n]\alpha = 0.995$

3.4.2 FFT Spectral Analysis and Windowing

A Hann window is applied before each spectral transform to reduce leakage. The fast DSP path uses a 256-point FFT on each 100 ms block for speech-band energy and spectral-flux estimation. Separately, the CNN preprocessing path uses a 512-point FFT with a 320-sample hop to form the 49 × 64 Log-Mel tensor. A-weighting is implemented as a time-domain three-stage biquad filter before the loudness calculation.

$$w(n) = 0.5 * (1 - cos\left(\frac{2\pi n}{N-1}\right))$$

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}, \quad k = 0,1,\dots,N-1$$

3.4.3 A-Weighting and Calibration

A-weighting is applied so that the measured level better follows human hearing perception. The uncalibrated digital RMS value alone is not a valid physical sound level. Therefore, this project uses a calibration-safe relation in which the final calibration constant is obtained experimentally by comparing the device output against a reference sound-level meter.

$$L_{dBFS} = 20\,log_{10}(V_{RMS})$$
$$L_{Aeq,T} = 20\,log_{10}(V_{RMS,A}) + C_{cal}$$

Here, V_RMS,A is the A-weighted RMS value over the measurement interval, and C_cal is the physical calibration offset. This is more defensible than relying only on a fixed theoretical microphone sensitivity, because microphone mounting, enclosure geometry, local reflections and firmware gain all affect the final measured value.

3.5 State Transition Architecture

The system uses a finite-state machine based on continuous audio measurements. These states provide clear and stable feedback to the user. The system consists of the seven states given below:

- STATE_SMOOTH: The baseline quiet state, indicating a comfortable environment.
- STATE_PRE_WARNING: A low-intensity alert indicating minor sustained disturbance.
- STATE_WARNING: An alert denoting a noticeable sustained disturbance.
- STATE_LOUD: A state triggered when raw physical sound pressure level exceeds significant values.
- STATE_RECOVERY: An intermediate cooldown state.
- STATE_SPIKE: A short overlay triggered by sudden physical impacts such as claps or slams.
- STATE_MIC_ERROR: A state that denotes hardware fault.

The finite-state machine transitions are defined through a priority waterfall evaluated every 100 ms.

The table below specifies the formal mathematical entry conditions, anti-windup clamping limits, and exit hysteresis rules defining the system's priority.

Table 4: FSM Waterfall Priority Transition Rules

Priority	Target State	Mathematical Entry Condition	Anti-Windup Clamping Limit	Exit / Fall-Through Condition
1	STATE_MIC_ERROR	t_err >= 2.0 s (micBadMs >= 2000)	3100 ms (1.1 s exit headroom)	micOk = True (t_err -> 0 s)
2	STATE_LOUD	Volume ≥ 76.0 dBA for 2.5 s (loudMs ≥ 2500)	3300 ms (0.8 s exit headroom)	Volume < 72.0 dBA (counter decays below entry threshold)
3	STATE_WARNING	t_warn >= 3.0 s (warningMs >= 3000)	4500 ms (1.5 s exit headroom)	Score < 65.0 (t_warn -> 0 s)
4	STATE_PRE_WARNING	t_pre >= 1.5 s (preWarningMs >= 1500)	3000 ms (1.5 s exit headroom)	Score < 45.0 (t_pre -> 0 s)
5	STATE_RECOVERY	Score < 45.0 AND Volume < 72.0 dBA	*Not Clamped*	t_rec >= 6.0 s (Transitions to SMOOTH)

3.6 Temporal Decision Engine and Scenario Timing Diagrams

The decision engine combines real-time deterministic features from the DSP pipeline with the CNN confidence scores. To generate stable visual feedback, it evaluates a score and passes the output through multiple parallel leaky integrators with clamping and priority mechanisms.

3.6.1 Fused Disturbance Index and Weight Allocation

A continuous index between 0.0 and 100.0 called the Acoustic Disturbance score is calculated instead of relying on a single acoustic value. The score is calculated every 100 ms using a Weighted Linear Combination formula:

$$Score = 100.0 * clamp(W_L * L + W_S * S + W_P * P + W_I * I, 0.0,1.0)$$

Where,

- L is the normalized Loudness parameter.
- S is the smoothed Speech Dominance probability.
- P is the slow-moving Persistence parameter.
- I is the transient Impulsiveness parameter.
- W_L, W_S, W_P, and W_I represent the respective weights assigned to each metric.

In the proposed system, these weights are allocated as follows:

- W_L = 0.45 (Loudness Weight)
- W_S = 0.25 (Speech Weight)
- W_P = 0.20 (Persistence Weight)
- W_I = 0.10 (Impulsiveness Weight)

The specific allocation of these weights is grounded in psychoacoustic and cognitive principles:

- **Loudness (45%):** The main reason for environmental sound load can be attributed to physical volume. This is why a weight of 45% is allocated to loudness itself. The parameter has a floor of 50dBA and a ceiling of 80dBA and is scaled to be between 0 and 1:

  $$L = clamp\big((LAeq_1s - 50.0)/30.0,0.0,1.0\big)$$

  A quiet room with a noise level below 50dBA contributes 0 points and a loud room with noise level above 80 dBA contributes 45 points to the score.

- **Speech Dominance (25%):** The human brain puts much more importance and weight on speech. The Irrelevant Speech Effect also shows that intelligible

background speech causes more disturbance as well as concentration loss even at lower sound levels. This is why a weight of 25% is assigned to speech effects rather than a steady background hum of the same decibel level.

- **Persistence (20%):** The persistence parameter represents the presence of a consistent steady noise. It works by acting as a low pass filter of instantaneous disturbance defined as the max between current loudness or speech values. The value essentially charges or discharges slowly as long as consistent noise is present or absent.
- **Impulsiveness (10%):** Impulsiveness aims to quantify sudden short noises, which may or may not be loud. These sounds may not affect the average sound level much but they are still an ever present factor in environmental disturbances. This parameter ensures they are registered in the final score.

3.6.2 CNN Semantic Integration and Probability Smoothing

The CNN model outputs softmax scores for three classes: stationary_background, speech_dominant and non_speech_activity. A software-level Exponential Moving Average with $\alpha = 0.40$ is applied to the speech probability before it is used as the S parameter in the score.

$$S_{smoothed}[t] = (1.0 - alpha)\, S_{smoothed}[t-1] + alpha\, S_{raw}[t]$$

3.6.3 Three-Parameter Envelope Spike Detection

Because the fused score does not fully account for sudden short sound events, a separate SPIKE overlay is implemented entirely through a DSP envelope detector. A 100 ms audio block is classified as a spike only if it satisfies three criteria at the same time:

1. Peak >= 78 dB (Ensures the noise is physically loud enough).
2. Crest Factor >= 10 dB (Calculated as Peak minus unweighted RMS).
3. Delta-above-Baseline >= 20 dB (Calculated as Peak minus the slow-moving, adaptive room baseline).

If a spike is detected, the SPIKE overlay is shown for 350 ms and uses a 300 ms clear-to-rearm interval.

3.6.4 Temporal Leaky Integrators with Hysteresis Headroom

State changes are facilitated by parallel millisecond counters associated with four conditions (preWarningMs, warningMs, loudMs and micBadMs). Every 100 ms, each counter increases by 100 ms while its condition is true and decreases by 100 ms when the condition clears. Anti-windup limits cap the counters above their trigger times and provide hysteresis headroom. A state falls through once its counter drops below the relevant entry threshold.

- **preWarningMs:** Entry threshold is 1500 ms. Capped at 3000 ms
- **warningMs:** Entry threshold is 3000 ms. Capped at 4500 ms
- **loudMs:** Entry threshold is 2500 ms. Capped at 3300 ms
- **micBadMs:** Entry threshold is 2000 ms. Capped at 3100 ms

3.6.5 State Transition Timing Diagrams

- **Scenario 1 – Standard Score Escalation and Stepped Decay. This diagram visualizes the pre-warning and warning display mechanisms.**

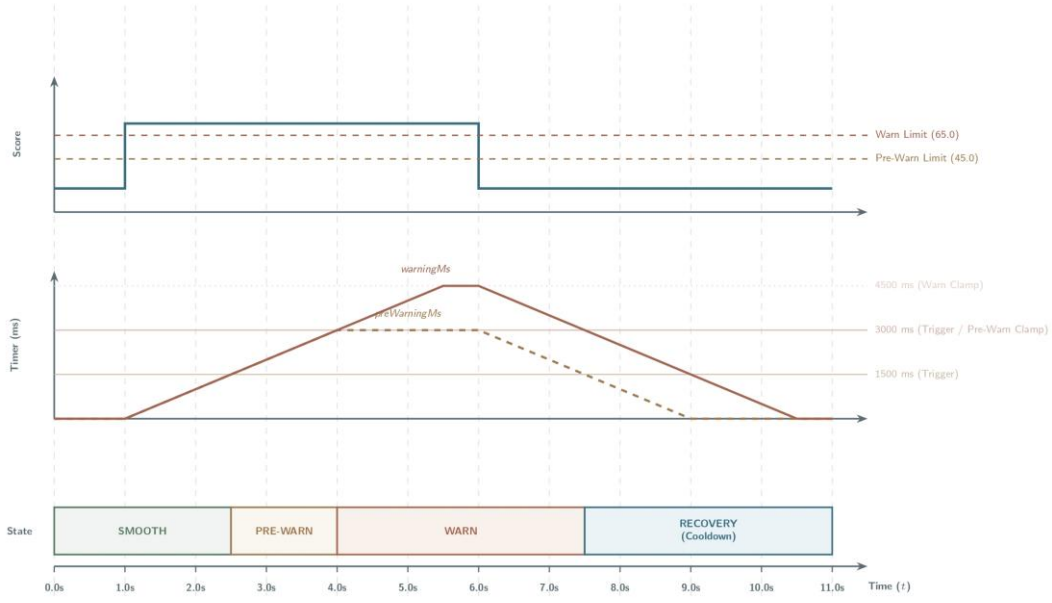


Figure 2: Standard Score Escalation and Stepped Decay

- **Scenario 2 – Loud State. This diagram shows the operation of the LOUD state.**

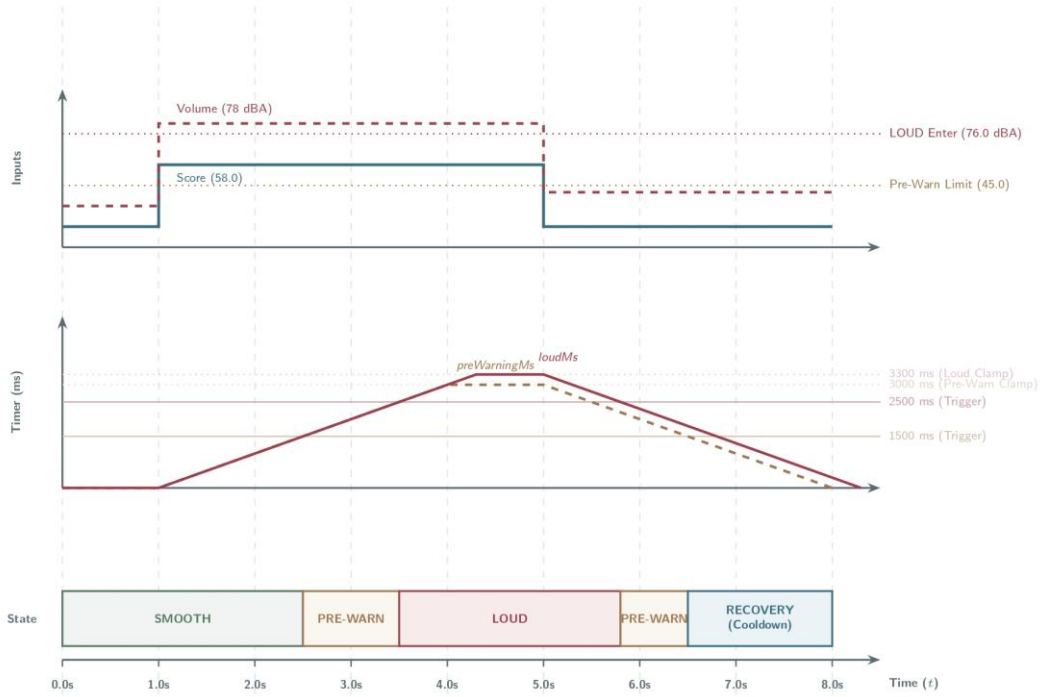


Figure 3: Loud State timing diagram

- **Scenario 3 – Hardware Fault Lockout. This diagram demonstrates the absolute priority of the hardware diagnostic.**

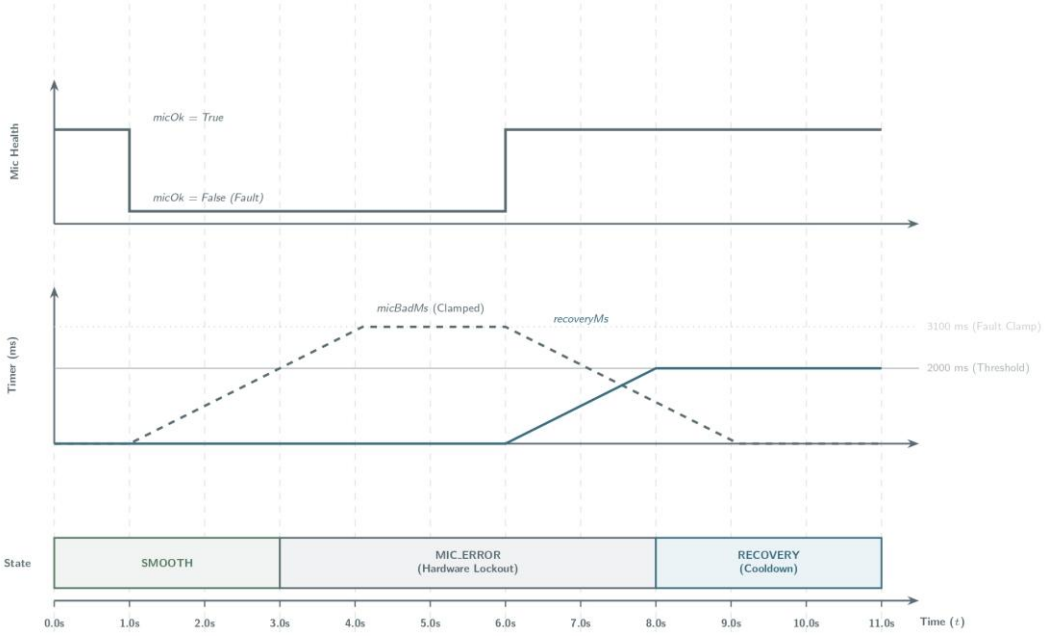


Figure 4: Hardware Fault Lockout Timing Diagram

3.7 Dataset Preparation and Machine Learning Training

To deploy a highly accurate, lightweight classifier on the ESP32-S3 microcontroller, the training pipeline utilizes a curated environmental sound dataset and a knowledge distillation framework.

3.7.1 The Environmental Sound Dataset

The dataset constructed for this project consists of 12,466 verified 1-second WAV audio clips standardized to 16 kHz mono PCM format. To prevent accidental random split leakage during iteration, all clips are partitioned into fixed, reproducible splits within the source repository.

The baseline distribution of the extracted clips across the three targeted classes is structured as follows:

- **Train (8,750 clips total):** 1,936 clips of stationary_background, 2,388 clips of speech_dominant, and 4,426 clips of non_speech_activity.
- **Validation (1,840 clips total):** 415 clips of stationary_background, 512 clips of speech_dominant, and 913 clips of non_speech_activity.
- **Test (1,876 clips total):** 415 clips of stationary_background, 512 clips of speech_dominant, and 949 clips of non_speech_activity.

Because the baseline training split contains a class imbalance (with non_speech_activity being the majority class), the pipeline applies deterministic oversampling during student training. Minority classes are oversampled to match the majority class count, resulting in a balanced training dataset of 13,278 total training samples (exactly 4,426 samples per class).

To improve generalization on the edge, the following data augmentation techniques are applied dynamically to the training batches:

- **SpecAugment:** Applies a single time mask of width up to 7 frames and a single frequency mask of width up to 9 Mel bins.
- **Time Shifting:** Moves the waveform temporally by -3 to +4 frames.
- **Feature Noise:** Adds random Gaussian noise with a standard deviation of 0.025 to the normalized features.

- **Mixup:** Blends pairs of training spectrograms and their target labels using a mixing ratio randomly sampled from a uniform distribution [0.70, 1.0].

3.7.2 Confusion Matrix Analysis

Following training under the guidance of the YAMNet teacher model, the student model is evaluated on the untouched test split in both its original high-precision Float32 format and its fully quantized, integer-only INT8 format.

Full integer quantization (PTQ) was enforced using a representative dataset of 250 samples per class to calibrate the dynamic ranges of all intermediate activation tensors. Both the input and output tensors are forced to signed 8-bit integers (int8), reducing the model size from 183.07 KB to 77,384 bytes (about 75.6 KiB).

The classification performance metrics on the test split are summarized below:

- **Float32 Student Model:** Test Accuracy = 89.77%, Balanced Accuracy = 88.50%, Macro-F1 = 88.90%.
- **Full INT8 Quantized Model:** Test Accuracy = 89.82%, Balanced Accuracy = 88.61%, Macro-F1 = 88.95%.

The confusion matrices below illustrate the high classification parity achieved between the floating-point and quantized models on the untouched test split (1,876 samples):

Table 5: Confusion Matrix before quantization

True Class / Predicted Class	stationary_ background	speech_ dominant	non_speech_ activity
stationary_background	323	3	89
speech_dominant	4	492	16
non_speech_activity	68	12	869

Table 6: Confusion Matrix after quantization

True Class / Predicted Class	stationary_ background	speech_ dominant	non_speech_ activity
stationary_background	324	4	87
speech_dominant	4	493	15
non_speech_activity	69	12	868

3.8 Output Behavior and UI Design

The planned physical output system is intentionally designed to avoid becoming another source of noise. The circular 1.28-inch TFT display will show the main state icon, dBA value, Wi-Fi status, battery or charging status, time, temperature and quiet streak. The 0.91-inch OLED will provide a compact debug/status view, while the LED

ring will provide ambient feedback through peripheral vision. The submitted firmware currently provides the state and diagnostic information through its local web dashboard.

In SMOOTH state, the planned LED ring will show a slow soothing breathing pattern transitioning between soft green and deep blue. In PRE-WARNING, a static low-intensity amber hint will appear. In WARNING, an amber melting animation will show sustained disturbance. In LOUD, red pulsing will indicate sustained loud noise. In SPIKE OVERLAY, a red ripple will appear for 350 ms and then return to the underlying state. The optional speaker or buzzer remains secondary and short because the project is designed primarily as a visual acoustic indicator.

3.9 Privacy Boundary Architecture

A core design philosophy of this project is the implementation of a strict Privacy by Design framework. Traditional IoT acoustic monitoring systems often stream raw audio to remote servers for feature extraction and inference, exposing users to potential eavesdropping, data leaks and biometric voice harvesting. This project isolates all audio processing within the ESP32-S3 volatile-memory boundary.

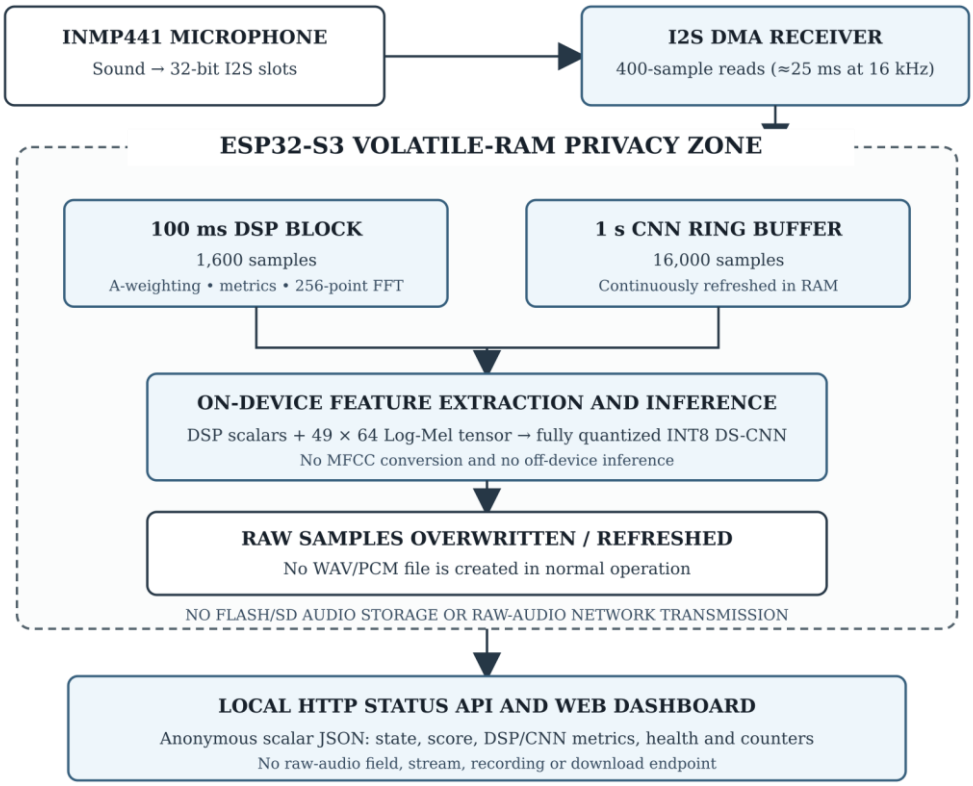


Figure 5: Privacy boundary architecture

Raw PCM samples from the INMP441 microphone are ingested exclusively into volatile RAM through the I2S DMA receiver. These samples are processed immediately to extract mathematical abstractions such as LAeq, FFT-derived metrics, speech-band energy and Log-Mel features. The raw samples are then overwritten by subsequent audio. No routine in the submitted firmware writes raw audio to flash memory, an SD card or a network packet.

The local JSON status API exposes only anonymous scalar variables such as the state, disturbance score, DSP metrics, CNN probabilities, microphone-health metrics, event counters, network status and uptime. It exposes no raw-audio field, stream, recording or download endpoint. MQTT-based remote telemetry remains planned and will follow the same scalar-only privacy boundary.

3.10 Instrumentation Tools

Table 7: Instrumentation Tools

Tool	Purpose
Arduino IDE / PlatformIO	Firmware development, deployment and serial debugging
Arduino-ESP32 / ESP-IDF APIs	I2S DMA, FreeRTOS, Wi-Fi and hardware-level control
Python / Keras / TensorFlow	Dataset processing, CNN training, validation and TFLite conversion
TensorFlow Lite Micro / Chirale TFLite	Microcontroller inference engine for the fully quantized INT8 model
Display and LED libraries (planned)	Circular TFT, OLED and WS2812B animation rendering
WebServer/HTTP; MQTT, Node-RED and Grafana (planned)	Current local status dashboard and planned remote telemetry/trend visualization
Reference sound-level meter	Calibration and comparison of LAeq readings
3D printing / enclosure tools	Product-like enclosure and hardware integration

3.11 Dashboard Instrumentation and Diagnostic Telemetry

3.11.1 Spectral Flux

Spectral Flux is a measure of the rate of change of the spectral envelope over consecutive blocks. It evaluates how quickly the frequency distribution of the room's sound field is shifting.

During the 256-point fast spectral analysis, the power spectrum is normalized relative to the total power of the block. The spectral flux is then calculated using the Euclidean distance between the normalized bins of the current block and the previous block:

$$F_{flux} = \sqrt{\sum_{k=0}^{128} (N_k[t] - N_k[t-1])^2}$$

where:

- N_k[t] is the normalized power spectrum magnitude of frequency bin k at the current block t.
- N_k[t-1] is the normalized power spectrum magnitude of frequency bin k at the previous block t-1.
- k represents the frequency bins up to the Nyquist limit (128 bins for a 256-point FFT).

Steady-state signals have a spectral flux close to 0.0 because their spectra change little from block to block. Dynamic, highly active sounds have higher spectral-flux values.

3.11.2 Clipping Percentage (Clip Percent)

Saturation or clipping occurs when the input sound pressure level exceeds the limits of the microphone. The DSP pipeline tracks the amplitude of every input sample within the 100 ms block. A sample is flagged as a clipping candidate if its absolute value is equal to or greater than 32,000 counts:

*Clip_Percent = (Count(abs(sample) >= 32000) / 1600) * 100.0*

3.11.3 Zeroes Percentage (Zero Percent)

The DSP pipeline monitors the absence of signal activity by counting the number of samples in the 100 ms block that fall below a noise floor threshold of 4 digital counts:

*Zero_Percent = (Count(abs(sample) < 4) / 1600) * 100.0*

A Zero_Percent value above 80.0% denotes an open-circuit failure, a loss of I2S master clock line, or a disconnected data pin.

3.11.4 Microphone Quality Index (Mic Quality)

To compress these hardware diagnostics into an intuitive health indicator, the decision engine evaluates a real-time index between 0% and 100% on every 100 ms block.

The Microphone Quality Index is calculated as:

$$Mic_Quality = 100.0$$
$$if\ (Zero_Percent > 80.0)\ then\ Mic_Quality = Mic_Quality - 65.0$$
$$if\ (Clip_Percent > 2.0)\ then\ Mic_Quality = Mic_Quality - 35.0$$
$$if\ (A_Weighted_dBFS < -80.0)\ then\ Mic_Quality = Mic_Quality - 45.0$$
$$Mic_Quality = clamp(Mic_Quality, 0.0, 100.0)$$

where:

A_Weighted_dBFS is the uncalibrated A-weighted energy relative to full scale.

3.11.5 Signal Above Room (Signal-to-Noise Ratio)

The Signal-to-Noise Ratio (SNR) represents the signal level relative to the baseline environment. It is calculated every 100 ms from the difference between calibrated A-weighted energy and the adaptive baseline.

$$SNR_dB = max(0.0, LAeq_1s - Baseline_dBA)$$

3.11.6 Speech-Band Ratio

The Speech-Band Ratio measures the portion of audio energy concentrated in the human vocal-frequency range. It is calculated as:

$$Speech_Band_Ratio = Speech_Power / Total_Power$$

Where:

- Speech_Power is the sum of the normalized spectral power bins from index 5 to index 54.
- Total_Power is the sum of all 128 normalized power bins up to the Nyquist limit (8 kHz).

3.11.7 System Uptime

Uptime is a system-level diagnostic parameter that tracks the continuous elapsed operational time of the microcontroller since its last boot sequence or hardware reset.

The parameter is calculated directly from the internal system clock of the microcontroller:

$$Uptime_s = millis() / 1000$$

Where:

- millis() returns the number of milliseconds elapsed since the ESP32-S3 began executing the active firmware.

3.11.8 Room Baseline

The Room Baseline represents the slowly updating, adaptive background noise floor of the environment, measured in A-weighted decibels (dBA). It acts as the local "0-point" of the room's continuous soundscape, allowing the system to measure relative acoustic disturbances rather than just absolute levels.

The baseline is updated recursively only when the room is relatively stable. Every 100 ms execution tick, if the current 1-second loudness (LAeq_1s) is less than the current baseline plus 6.0 dBA and the signal is non-impulsive (Crest Factor is below 16.0 dB), the baseline is updated using a slow Exponential Moving Average (EMA):

$$Baseline_dBA[t] = 0.998 * Baseline_dBA[t-1] + 0.002 * LAeq_1s$$

Where:

- *Baseline_dBA[t] is the updated room baseline for the current block.*
- *Baseline_dBA[t-1] is the baseline from the previous block.*
- LAeq_1s is the current 1-second A-weighted equivalent sound level.

3.11.9 Crest Factor

The Crest Factor is a physical ratio that measures the "peakedness" of the acoustic signal. It represents the difference between the absolute peak pressure level and the unweighted root-mean-square (RMS) energy of the 100 ms audio block, expressed in decibels.

The parameter is calculated on every 100 ms execution block using the uncalibrated peak and unweighted RMS measurements:

$$Crest_dB = Peak_dB - RMS_dB$$

Where:

- Peak_dB is the maximum absolute sample amplitude captured in the 100 ms block, converted to a decibel scale and offset by the physical calibration constant:
  Peak_dB = 20 * log10(abs_peak_amplitude / 32768.0) + C_cal
- RMS_dB is the unweighted RMS energy of the 100 ms block converted to decibels and offset by the same calibration constant:
  RMS_dB = 10 * log10(block_unweighted_power) + C_cal

4. EXPECTED OUTCOME AND VALIDATION PLAN

The project aims to deliver a working product-style prototype that shows real-time audio classification with visual feedback. The prototype will not be a certified legal sound meter; it will act as a behavioral acoustic comfort indicator that helps occupants self-regulate through calm visual feedback.

At the current progress stage, the submitted build already implements I2S audio capture, DSP diagnostics, the fully quantized INT8 DS-CNN, hybrid fusion, state timing, stored calibration, labelled simulation controls and a local HTTP dashboard. The remaining work focuses on physical output integration, enclosure completion, threshold calibration and controlled real-world validation.

- A product-style prototype with circular RGB LCD, OLED, LED ring, ESP32-S3, INMP441 microphone, rechargeable battery, speaker option and enclosure.
- Real-time A-weighted measurement, speech-band ratio, peak detection, quiet streak and audio-state differentiation.
- Smooth visual state animations for SMOOTH, PRE-WARNING, WARNING, LOUD, RECOVERY and SPIKE OVERLAY.
- Current dBA, state, score, event count, spike count and diagnostic metrics shown on the local dashboard, with Node-RED/Grafana trend visualization planned as an extension.
- Validation report with labeled samples, confusion matrix, precision/recall/F1 metrics and calibration comparison with a reference meter.

4.1 Real-World Validation Scenarios

Table 8: Validation Scenario Matrix

Scenario ID	Tested Noise Source	Physical Range	Target Class	Expected Visual Feedback	Expected Dashboard Log
VAL-01	Standard conversation	58-66 dBA	speech_dominant	Amber melt animation after 3 seconds	WARNING
VAL-02	Quiet study baseline	< 50 dBA	Smooth background	Blue/green breathing wave	SMOOTH
VAL-03	AC unit / ceiling fan	52-56 dBA	stationary_background	Remain blue/green breathing	SMOOTH
VAL-04	Transient clapping / object drop	≥ 78 dBA peak	DSP-only spike	Immediate red ripple overlay for 350 ms	SPIKE
VAL-05	Group shouting / loud room	≥ 76 dBA	DSP-only loud condition	Pulse-red warning glow after 2.5 seconds	LOUD
VAL-06	Speech stops after warning	Below 53 dBA stable	Smooth background	Fade back after 6 seconds	RECOVERY -> SMOOTH

4.2 Physical Measurement Accuracy Calibration

The LAeq readings will be calibrated by placing the AuraSync prototype and a commercial reference sound-level meter side-by-side. A steady 1 kHz reference tone will be played from an external speaker, and the level will be adjusted from 50 dBA to 80 dBA in 5 dBA increments. For each level, the raw uncalibrated reading from the INMP441 pipeline and the reference meter reading will be recorded. The calibration offset C_cal will be calculated from the difference between the reference reading and the raw A-weighted RMS reading.

$$C_{cal} = Reading_{reference} - 20\,log_{10}(V_{RMS,A})$$

After calibration, the device will be tested again using fan noise, speech, clap, object drop and loud talking. The expected result is not legal metrology accuracy, but stable acoustic indication within a practical academic prototype tolerance.

4.3 Machine Learning Validation

The CNN/TinyML model has been evaluated on the separate test set using confusion matrices, class-wise precision, recall and F1-score. Further validation will focus on live-device generalization, especially false positives between stationary_background and speech_dominant. The DSP-only LOUD and SPIKE outputs will be validated separately using a calibrated reference meter and controlled acoustic events.

5. PROJECT SCHEDULE

The project is planned for six weeks of implementation after topic approval. Hardware and software tasks can run in parallel to reduce risk and allow early demonstration.

Table 9: Project timeline

Time Period	Major Activity	Main Tasks	Expected Output
Week 1	Hardware engineering and HAL setup	Procure components, test SPI/I2C buses, implement basic I2S DMA audio capture	Clean digital audio samples successfully printed to serial plotter
Week 2	DSP and decibel calibration	Implement RMS, peak, A-weighting, baseline smoothing and reference-meter calibration	Real-time dBA readings calibrated against reference meter
Week 3	CNN training and quantization	Prepare dataset, train custom CNN, perform INT8 quantization and memory testing	Model compiles and runs in ESP32-S3 test firmware
Week 4	Hybrid state engine and UI	Implement thresholds, timers, hysteresis, LCD/OLED/LED animations	Flicker-free local state transitions
Week 5	Telemetry and dashboard	Implement MQTT publisher, broker, Node-RED/Grafana panels and JSON logging	Dashboard displays dBA, states, event counts and trends
Week 6	Integration and finalization	Assemble enclosure, run validation scenarios, complete report and prepare demo	Fully functional prototype ready for defense

The schedule is feasible because the rule-based classifier can be implemented first while TinyML training and enclosure polishing continue in parallel. This reduces the risk of having no working prototype even if model tuning takes longer than expected.

6. PROJECT BUDGET

The estimated budget is prepared for a functional academic prototype. Prices may vary depending on local availability and component quality. The reference sound-level meter may be borrowed from the laboratory to reduce total cost.

Table 10: Project budget

Component Description	Quantity	Unit Cost (NPR)	Total Cost (NPR)
ESP32-S3 development board with PSRAM	1	1,200	1,200
INMP441 I2S omnidirectional microphone module	2	600	1,200
GC9A01 1.28-inch circular TFT LCD (SPI)	1	1,400	1,400
SSD1306 0.91-inch OLED display (I2C)	1	500	500
WS2812B RGB LED ring	1	500	500
3.7 V LiPo battery, TP4056 charger and BMS	1 set	1,500	1,500
DHT22 / temperature-humidity sensor	1	400	400
Speaker/buzzer and small components	1 set	500	500
Enclosure, stand and wiring harness	1 set	1,000	1,000
Reference sound-level meter	Borrowed / optional	0	0
Contingency fund	1	1,800	1,800
Estimated Total			10,000

The cost can be reduced if the reference sound-level meter is borrowed from the laboratory. Without the calibration meter, the core prototype remains within a practical academic range, but calibration quality will improve if a reference meter is used during validation.

7. FEASIBILITY ANALYSIS

The proposed system is feasible from technical, economic, operational, privacy and academic viewpoints. ESP32-S3 has sufficient capability for I2S sampling, fixed-size FFT processing, compact TinyML inference and Wi-Fi telemetry. The submitted firmware uses a dedicated audio task and a short, non-blocking web-service loop to preserve real-time operation. A custom compact model is preferred over a large general audio model to keep inference fast and offline.

Table 11: Feasibility analysis summary

Feasibility Aspect	Analysis
Technical feasibility	ESP32-S3, INMP441, displays, LED ring and MQTT libraries are compatible and available. Dual-core task separation supports real-time operation.
Economic feasibility	Most components are low-cost and locally available for an academic prototype. Total estimated budget is around NPR 10,000.
Time feasibility	The six-week plan is realistic because rule-based classification can work before TinyML optimization is complete.
Operational feasibility	The visual feedback is non-judgmental, simple to understand and suitable for classrooms, libraries and shared workspaces.
Privacy feasibility	Raw audio is not stored or transmitted; only anonymous features and metrics are published.
Risk control	Calibration drift, false alarms and power issues can be reduced through reference calibration, smoothing, hysteresis and stable power design.

7.1 Limitations

The first prototype will require threshold tuning for different rooms and will not replace a certified sound-level meter. Model accuracy will depend on dataset diversity and how closely the training data matches the final microphone, enclosure and room conditions. The local dashboard requires a Wi-Fi-capable client, although all DSP, inference and state decisions operate offline. The device is designed for acoustic comfort indication, not for legal, forensic or occupational-noise certification.

7.2 Risk Management

The main technical risk is false triggering caused by non-speech noise. This is controlled by using speech-band ratio and CNN confidence in addition to dBA thresholds. The second risk is missed spike detection due to the one-second CNN window. This is controlled by the fast DSP peak detector, which reacts within the 100 ms update path. The third risk is user annoyance due to flickering output. This is controlled through confirmation timers and recovery hysteresis.

7.3 Future Enhancements

Future versions can include a custom PCB, improved acoustics, IP-rated enclosure design, admin-selectable noise policy modes, secure MQTTS, multi-device dashboards, over-the-air firmware updates and improved edge AI models trained on larger device-specific datasets. Additional adaptive calibration can also be introduced so that the device automatically learns room baselines over time while still preserving privacy.

REFERENCES

[1] W. Ellermeier and K. Zimmer, "Individual differences in susceptibility to the irrelevant speech effect," The Journal of the Acoustical Society of America, vol. 102, no. 5, pp. 2685-2691, Nov. 1997.

[2] M. Haka, A. Haapakangas, J. Keranen, J. Hakala, E. Keskinen, and V. Hongisto, "Performance effects and subjective disturbance of speech in acoustically different office types - a laboratory experiment," Indoor Air, vol. 19, no. 6, pp. 454-467, 2009.

[3] J. Radun, J. Ljung, and V. Hongisto, "The effects of irrelevant speech on physiological stress, cognitive performance, and subjective experience - Focus on heart rate variability," International Journal of Psychophysiology, vol. 200, p. 112352, Apr. 2024.

[4] E. Lombard, "Le signe de l'elevation de la voix," Annales des Maladies de l'Oreille, du Larynx, du Nez et du Pharynx, vol. 37, pp. 101-119, 1911.

[5] P. Bottalico, I. Ipsaro Passione, S. Graetzer, and E. J. Hunter, "Evaluation of the starting point of the Lombard effect," Acta Acustica united with Acustica, vol. 103, no. 1, pp. 169-172, 2017.

[6] P. Bottalico, R. N. Piper, and B. Legner, "Lombard effect, intelligibility, ambient noise, and willingness to spend time and money in a restaurant amongst older adults," Scientific Reports, vol. 12, no. 1, p. 6549, 2022.

[7] J. Whitlock and G. Dodd, "Classroom acoustics: Personal and environmental factors," New Zealand Journal of Audiology, vol. 16, no. 1, pp. 18-29, 2008.

[8] L. Zelem, V. Chmelik, C. Glorieux, and M. Rychtarikova, "Correlation of room acoustic parameters and noise level in eating establishments," Acta Acustica, vol. 7, p. 32, 2023.

[9] ISO 22955:2021, Acoustics - Acoustic quality of open office spaces, International Organization for Standardization, Geneva, Switzerland, 2021.

[10] ANSI/ASA S12.60-2010/Part 1, American National Standard Acoustical Performance Criteria, Design Requirements, and Guidelines for Schools, Part 1: Permanent Schools, Acoustical Society of America, Melville, NY, USA, 2010.

[11] K. S. Pearsons, R. L. Bennett, and S. Fidell, "Speech levels in various environments," U.S. Environmental Protection Agency, Tech. Rep. EPA-600/1-77-025, May 1977.

[12] A. N. Rimell, N. J. Mansfield, and G. S. Paddan, "Design of digital filters for frequency weightings required for risk assessments of workers exposed to noise," Industrial Health, vol. 53, no. 1, pp. 21-27, 2015.

[13] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," Mathematics of Computation, vol. 19, no. 90, pp. 297-301, Apr. 1965.

[14] J. Ramirez, J. M. Gorriz, and J. C. Segura, "Voice Activity Detection: Fundamentals and Speech Recognition System Robustness," in Robust Speech Recognition and Understanding, M. Grimm and K. Kroschel, Eds. Vienna, Austria: I-Tech Education and Publishing, 2007, ch. 1, pp. 1-22.

[15] S. Hershey et al., "CNN architectures for large-scale audio classification," in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), New Orleans, LA, USA, 2017, pp. 131-135.

[16] G. Marques and R. Pitarma, "A real-time noise monitoring system based on IoT for enhanced acoustic comfort and occupational health," IEEE Access, vol. 8, pp. 13974-13985, 2020.

[17] M. Hammad, A. M. Al-Masri, M. Al-Hammadi, and B. S. Al-Radaideh, "An unsupervised TinyML approach to detection of urban noise anomalies," Internet of Things, vol. 22, art. no. 100723, 2023.

[18] P. Zadpe, S. Kulkarni, and R. V. Kshirsagar, "Lightweight voice authentication for IoT devices using MFCC and CNN on edge hardware," International Journal for Research in Applied Science and Engineering Technology, vol. 13, no. 4, pp. 235-242, Apr. 2025.

[19] G. Cerutti, R. Prasad, A. Brutti, and E. Farella, "Compact recurrent neural networks for acoustic event detection on low-energy low-complexity platforms," IEEE Journal of Selected Topics in Signal Processing, vol. 14, no. 4, pp. 786-797, May 2020.

[20] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[21] Espressif Systems, "ESP32-S3 Series Datasheet," Espressif Systems, Shanghai, China, 2023.

[22] TDK InvenSense, "INMP441 Omnidirectional Microphone with I2S Digital Output Datasheet," TDK Corporation.

[23] Real Time Engineers Ltd., The FreeRTOS Kernel Developer Guide, 2022. [Online]. Available: https://www.freertos.org/.

[24] Espressif Systems, "ESP-DL: Deep Learning Library for Espressif SoCs," 2023. [Online]. Available: https://github.com/espressif/esp-dl.

[25] M. F. Usmani, "MQTT Protocol for the IoT - Review Paper," International Journal of Computer Applications, vol. 183, no. 39, pp. 17-22, Jun. 2021.

[26] N. Radwan and J. Alves-Foss, "MQTT in focus: Understanding the protocol and its recent advancements," 2024.

[27] OpenJS Foundation, "Node-RED: Low-code programming for event-driven applications," 2024. [Online]. Available: https://nodered.org/.

[28] S. S. Gill et al., "Edge AI: A taxonomy, systematic review and future directions," Cluster Computing, vol. 28, no. 1, p. 18, 2025.